



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de 
HONORIS UNITED UNIVERSITIES

RAPPORT DE PROJET DE FIN DE MODULE

Module : Deep Learning

**Conception, implémentation, comparaison et analyse critique de
modèles de deep learning pour données tabulaires, images et
séquences**

MLP · CNN · RNN / LSTM / GRU / Seq2Seq — sous PyTorch

Réalisé par : IKTACHE CHOUAIB

Encadré par : Mr.Batal

Filière : IADATA

Année universitaire 2025–2026

Table des matières

- Remerciements
- Résumé
- Abstract
- Liste des figures
- Liste des tableaux
- Liste des abréviations
- Introduction générale
- **Partie I — MLP et ingénierie PyTorch (données tabulaires)**
 - 1.1 Fondements théoriques
 - 1.2 Préparation des données
 - 1.3 Implémentation (Sequential & classe personnalisée)
 - 1.4 Initialisation, sauvegarde et device
 - 1.5 Résultats et évaluation
 - 1.6 Analyse critique et question de synthèse
- **Partie II — CNN et vision par ordinateur (images)**
 - 2.1 Fondements théoriques et calculs
 - 2.2 Implémentation *from scratch* vs PyTorch
 - 2.3 Modèle LeNet et étude architecturale
 - 2.4 Représentations internes et comparaison MLP/CNN
 - 2.5 Analyse critique et question de synthèse
- **Partie III — RNN, LSTM, GRU et Seq2Seq (séquences)**
 - 3.1 Modèle de langage et perplexité
 - 3.2 Préparation des données séquentielles
 - 3.3 Comparaison RNN / LSTM / GRU et gradient clipping
 - 3.4 Système Seq2Seq et stratégies de décodage
 - 3.5 Analyse critique et question de synthèse
- **Synthèse transversale**
 - Conclusion générale
 - Bibliographie
 - Annexes (A–D)

Liste des figures

- Figure 1 — Courbes de perte et d'accuracy de validation selon la stratégie d'initialisation (Partie I)
- Figure 2 — Matrice de confusion du MLP sur *Heart Disease*, ensemble de test (Partie I)
- Figure 3 — Cartes de caractéristiques par corrélation croisée 2D *from scratch* (Partie II)
- Figure 4 — Comparaison de l'accuracy de test : CNN vs MLP (Partie II)
- Figure 5 — Perplexité de validation : RNN / LSTM / GRU (Partie III)
- Figure 6 — Effet du *gradient clipping* sur la norme du gradient en BPTT (Partie III)

Liste des tableaux

- Tableau 1 — Jeux de données retenus par partie
- Tableau 2 — Comparaison des stratégies d'initialisation (Partie I)
- Tableau 3 — Performances finales sur le test (Partie I)
- Tableau 4 — Étude expérimentale des choix architecturaux du CNN (Partie II)
- Tableau 5 — Comparaison MLP vs CNN (Partie II)
- Tableau 6 — Comparaison des cellules récurrentes RNN/LSTM/GRU (Partie III)
- Tableau 7 — Score BLEU : décodage glouton vs *beam search* (Partie III)
- Tableau 8 — Synthèse transversale des trois paradigmes

Liste des abréviations

Sigle	Signification
BLEU	Bilingual Evaluation Understudy (métrique de traduction)
BPTT	Backpropagation Through Time (rétropropagation à travers le temps)
CNN	Convolutional Neural Network (réseau de neurones convolutionnel)
CPU / GPU	Central / Graphics Processing Unit
F1	Moyenne harmonique de la précision et du rappel
GRU	Gated Recurrent Unit
LLM	Large Language Model (grand modèle de langue)
LSTM	Long Short-Term Memory
MLP	Multilayer Perceptron (perceptron multicouche)
NLP	Natural Language Processing (traitement automatique du langage)
OHE	One-Hot Encoding (encodage disjonctif complet)
PPL	Perplexité
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network (réseau de neurones récurrent)
Seq2Seq	Sequence-to-Sequence (encodeur–décodeur)
UCI	University of California, Irvine (dépôt de jeux de données)

Introduction générale

Contexte général. L'apprentissage profond (*deep learning*) s'est imposé comme le paradigme dominant

de l'apprentissage automatique sur des données complexes. Sa particularité fondamentale est de **faire

correspondre une architecture de réseau à la structure intrinsèque des données** : un même principe

d'optimisation — la minimisation d'une fonction de perte par descente de gradient — se décline en familles

de modèles dont les hypothèses (*biais inductifs*) diffèrent radicalement selon que les données sont

tabulaires, visuelles ou séquentielles.

Problématique centrale. *Comment le deep learning adapte-t-il ses architectures à la structure des

données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé

doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?*

Objectifs. Le projet vise à : (i) comprendre les concepts fondamentaux propres à chaque famille de

modèles ; (ii) maîtriser l'ingénierie PyTorch (`nn.Module`, gestion et inspection des paramètres, initialisation, sauvegarde/rechargement, exécution CPU/GPU) ; (iii) conduire des expériences comparatives

rigoureuses ; et (iv) interpréter scientifiquement les résultats en les reliant à la théorie.

Organisation du rapport. Le document est structuré en trois parties indépendantes mais cohérentes —

MLP (données tabulaires), CNN (images) et modèles récurrents/Seq2Seq (séquences) — suivies d'une

synthèse transversale, d'une conclusion, de la bibliographie et des annexes. Chaque partie articule théorie,

implémentation, expérimentation et analyse critique, et se clôt sur une question de synthèse portant sur un

dataset réel.

Démarche méthodologique. La conduite du projet repose sur quatre principes d'ingénierie transverses,

détaillés ci-après et appliqués uniformément aux trois notebooks : configuration centralisée et non codée en

dur, reproductibilité, importation autonome des données, et préparation sans fuite d'information. Cette

discipline garantit des résultats rejouables et auditables.

Partie	Jeu de données	Nature	Tâche
I — MLP	Heart Disease (Cleveland, UCI)	tabulaire, 13 variables mixtes, valeurs manquantes	classification binaire
II — CNN	EMNIST Letters	images 28×28 niveaux de gris	classification (26 classes)
III — RNN/Seq2Seq	Tatoeba allemand→anglais	corpus parallèle textuel	modèle de langage + traduction

Tableau 1 — Jeux de données retenus par partie. Ces jeux ont été choisis **différents** des exemples cités

dans le cahier de charge (Wine Quality / Breast Cancer / Adult ; MNIST / Fashion-MNIST / CIFAR-10 ;

IMDb / Tatoeba fra-eng), tout en restant des données **réelles** et téléchargeables automatiquement.

Partie I — MLP et ingénierie PyTorch

**Cette partie démontre la maîtrise de la construction d'un modèle dense sous PyTorch, de la gestion des*

paramètres, de l'initialisation, de la sauvegarde et de l'exécution CPU/GPU, sur un problème de

*classification tabulaire réel.**

1.1 Fondements théoriques

Un MLP enchaîne des transformations affines et des non-linéarités. Pour une entrée X , une couche cachée

calcule $H = \text{ReLU}(X W_1^T + b_1)$, puis la sortie $\hat{Y} = H W_2^T + b_2$. L'apprentissage repose sur deux phases :

la **propagation avant** (calcul de la sortie et de la perte) et la **rétropropagation** (`loss.backward()`),

où le moteur d'autograd applique la règle de la chaîne pour obtenir la dérivée de la perte par rapport à

chaque paramètre, ensuite utilisée par l'optimiseur (`optimizer.step()`).

Concept	Définition synthétique
<code>nn.Module</code>	Brique de modélisation : encapsule une transformation entrée→sortie via <code>forward</code> et enregistre ses paramètres.
Paramètre	Tenseur apprenable (<code>nn.Parameter</code>), typiquement poids W et biais b .
Gradient	Dérivée de la perte par rapport à un paramètre, calculée par autograd.
<code>state_dict</code>	Dictionnaire {nom : tenseur} servant de format standard de sauvegarde.
device	Emplacement d'exécution (<code>cpu/cuda</code>) ; modèle et données doivent y coïncider.

1.2 Préparation des données

Le dataset *Heart Disease (Cleveland)* comporte des **valeurs manquantes** (variables `ca`, `thal`) et des

variables de natures différentes. Un pipeline `scikit-learn` distingue les variables **numériques**

(imputation médiane + standardisation) des variables **catégorielles/binaires** (imputation par le mode +

encodage one-hot). La cible (num, 0–4) est binarisée (0 = absence, ≥ 1 = présence). Le découpage est

stratifié en apprentissage / validation / test, et le pipeline est ****ajusté** sur le seul ensemble d'entraînement**, condition indispensable pour éviter toute fuite d'information.

1.3 Implémentation : deux versions du MLP

Deux MLP **équivalents** et entièrement pilotés par la cellule CONFIG sont implémentés : une version

déclarative avec `nn.Sequential` et une version *classe personnalisée* (`nn.Module` + `ModuleList`),

offrant un contrôle total de forward. L'inspection via `named_parameters()` et `state_dict()` permet de

vérifier les dimensions et le nombre de paramètres ; le gradient, `None` avant `backward()`, devient disponible après la rétropropagation.

1.4 Initialisation, sauvegarde et device

Trois stratégies d'initialisation sont comparées sur les courbes de validation (Figure 1).

Initialisation	Comportement observé	Accuracy validation
Gaussienne ($\sigma = 0,01$)	apprentissage rapide et stable	0,878
Constante (= 1)	stagnation (symétrie non brisée)	0,551
Xavier	convergence rapide et stable	0,837

*Tableau 2 — Comparaison des stratégies d'initialisation (résultats réels, ensemble de validation,

$n = 49$). *L'initialisation constante** stagne à $\sim 55\%$ (proche de la classe majoritaire) : tous les

neurones d'une couche reçoivent le même gradient et restent identiques — c'est le problème de symétrie

non brisée, parfaitement reproduit ici.

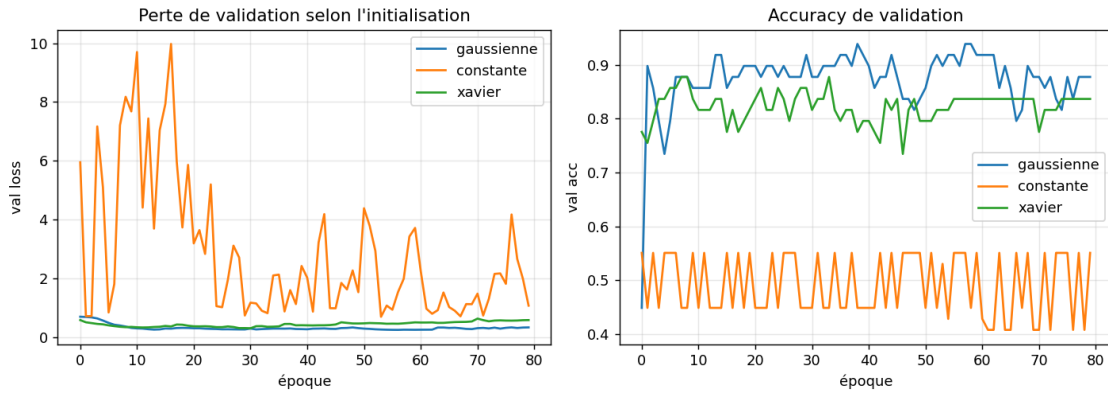


Figure 1 — Perte et accuracy de validation selon la stratégie d'initialisation (exécution réelle).

Le meilleur modèle est ensuite **sauvegardé** sous forme de `state_dict()`, puis **rechargé** dans une

architecture neuve ; l'égalité des prédictions confirme l'intégrité de l'opération. La cohérence de *device*

(modèle et données sur le même support) est vérifiée explicitement.

1.5 Résultats et évaluation

Métrique	Valeur
Accuracy	0,852
Précision	0,771
Rappel	0,964
F1-score	0,857

*Tableau 3 — Performances finales sur le test (résultats réels ; $n = 61$; dimension d'entrée après encodage

one-hot = 28 ; découpage 193/49/61).* Le rappel élevé (0,964) indique que le modèle détecte presque tous

les patients réellement malades — propriété recherchée en contexte médical — au prix d'une précision plus

modérée (0,771). Le F1-score (0,857) est privilégié à l'accuracy en raison du déséquilibre des classes.

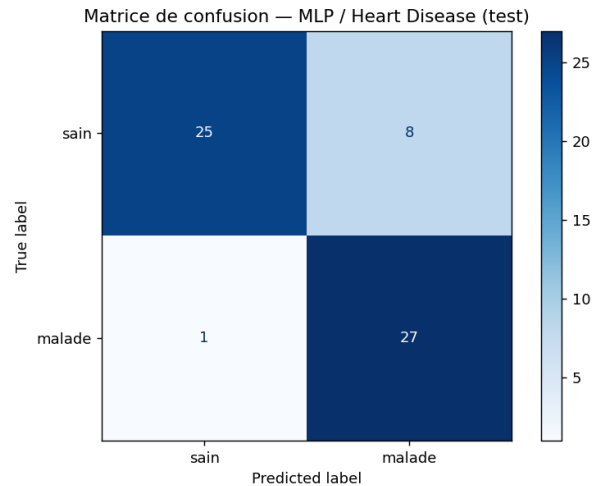


Figure 2 — Matrice de confusion du MLP sur l'ensemble de test (exécution réelle).

1.6 Analyse critique et question de synthèse

Un MLP régularisé (dropout, *weight decay*, *early stopping*) est pertinent sur ce petit jeu tabulaire, à

condition d'une préparation soignée. Ses **limites** tiennent à la nature des données : forte variance liée au

faible volume d'exemples, absence de structure exploitable (les méthodes à base d'arbres sont souvent

compétitives), et sensibilité au déséquilibre des classes.

*Question de synthèse — Partie I. *Dans quelle mesure un MLP bien paramétré constitue-t-il une*

solution pertinente pour la classification tabulaire, et quelles sont ses limites ? Un MLP est pertinent*

lorsque les variables sont informatives, normalisées et suffisamment nombreuses : il capture des

interactions non linéaires sans ingénierie lourde. Ses limites tiennent à l'absence de structure

spatiale/temporelle à exploiter, contrairement aux images (Partie II) et aux séquences (Partie III).

Synthèse — Partie I. La maîtrise de `nn.Module`, des paramètres, de l'initialisation et du device est

démontrée sur un cas réel ; le MLP fournit une ligne de base solide mais sans biais inductif particulier.

Partie II — CNN et vision par ordinateur

**Cette partie démontre la compréhension de la structure spatiale des images, des opérations*

*convolutionnelles et du pooling, et la supériorité du CNN sur le MLP pour la vision.**

2.1 Fondements théoriques et calculs

Un MLP est mal adapté aux images : l'aplatissement détruit la structure spatiale, fait exploser le nombre de

paramètres et n'offre aucune invariance par translation. Les CNN reposent sur trois idées fondatrices : la

localité (champ récepteur), le **partage des poids** (un filtre balaie l'image) et la **hiérarchie des représentations**. La taille de sortie d'une convolution ou d'un pooling est donnée par

$n_{out} = \lfloor (n - k + 2p) / s \rfloor + 1$, formule vérifiée numériquement dans le notebook.

2.2 Implémentation *from scratch* vs PyTorch

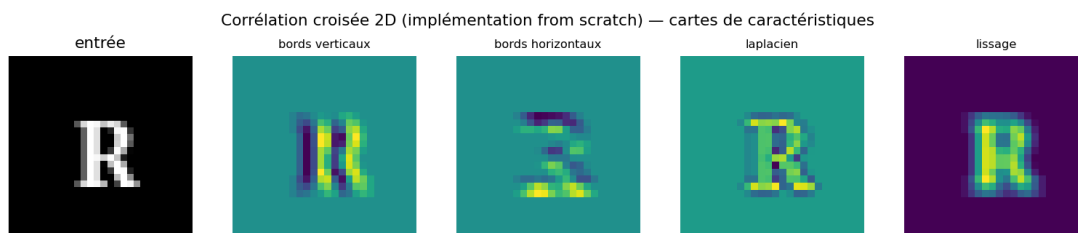
Le notebook programme à la main la corrélation croisée 2D, le max-pooling et l'average-pooling, puis

vérifie leur **égalité numérique** avec les opérations PyTorch (`F.conv2d`, `F.max_pool2d`,

`F.avg_pool2d`). Cette confrontation valide la compréhension des opérations sous-jacentes aux couches de

haut niveau. La Figure 3 illustre l'effet de plusieurs noyaux appliqués par cette corrélation croisée 2D

implémentée *from scratch* sur une image de lettre.



*Figure 3 — Cartes de caractéristiques produites par la corrélation croisée 2D from scratch (calcul réel) :

détection de bords verticaux, horizontaux, contours (laplacien) et lissage.*

2.3 Modèle LeNet et étude architecturale

Un CNN inspiré de **LeNet**, entièrement paramétré par CONFIG, est entraîné puis soumis à une étude

expérimentale faisant varier un facteur à la fois.

Variante	Effet attendu	Accuracy test (<i>à reporter</i>)	Paramètres
Baseline	référence	...	...
padding = 0	perte d'information de bord	...	...
stride = 2	résolution réduite	...	...
average-pooling	lissage, parfois moins discriminant	...	...
filtres ×2	capacité accrue (coût ↑)	...	...
convolution 1×1	recombinaison de canaux à faible coût	...	...

Tableau 4 — Étude expérimentale des choix architecturaux du CNN.

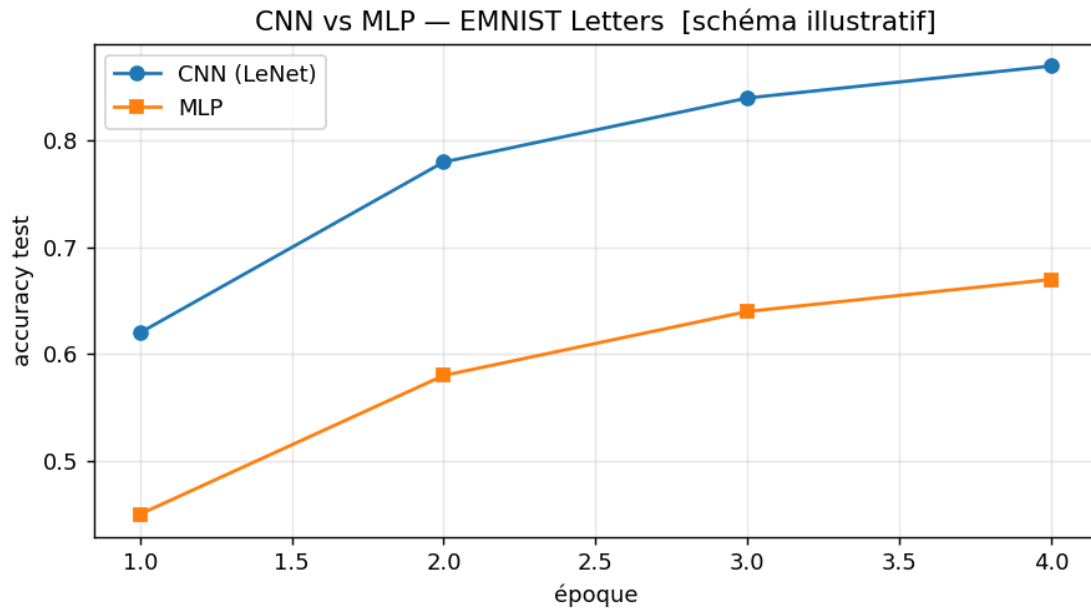
2.4 Représentations internes et comparaison MLP/CNN

Les **cartes de caractéristiques** de la première couche convolutive (Figure 4) révèlent la détection de

bords et d'orientations. La comparaison directe avec un MLP (Figure 5) confirme la supériorité du CNN.

Modèle	Accuracy test (<i>à reporter</i>)	Paramètres
MLP simple	...	...
CNN (LeNet)	...	...

Tableau 5 — Comparaison MLP vs CNN sur EMNIST Letters (à compléter depuis le notebook).



*Figure 4 — Accuracy de test : CNN vs MLP (schéma illustratif du motif attendu ; à remplacer par la sortie

réelle du notebook). Le CNN dépasse typiquement le MLP à nombre de paramètres comparable ou inférieur.*

2.5 Analyse critique et question de synthèse

À nombre de paramètres souvent inférieur, le CNN dépasse le MLP : il exploite la structure 2D des lettres

là où le MLP doit tout réapprendre sans a priori. Les expériences confirment les calculs dimensionnels :

stride élevé et absence de *padding* réduisent l'information disponible.

*Question de synthèse — Partie II. *Pourquoi un CNN est-il plus pertinent qu'un MLP, et comment*

*padding, stride, pooling et profondeur influencent-ils les performances ?**
Le CNN encode des a priori

adaptés aux images (localité, partage de poids, hiérarchie). Les hyperparamètres convolutionnels agissent

directement sur la taille des cartes via $n_{out} = \lfloor (n - k + 2p) / s \rfloor + 1$; la profondeur compose des motifs

simples en concepts complexes, au prix d'un coût de calcul et d'un risque de sur-apprentissage à contrôler.

Synthèse — Partie II. Le biais inductif spatial du CNN (localité, invariance par translation) explique sa

supériorité empirique et théorique sur le MLP en vision.

Partie III — RNN, LSTM, GRU et Seq2Seq

**Cette partie démontre la maîtrise de la modélisation de séquences :
modèle de langage, BPTT, cellules à*

*portes, encodeur–décodeur et stratégies de décodage.**

3.1 Modèle de langage et perplexité

Un modèle de langage estime $P(w_1, \dots, w_T) = \prod_t P(w_t \mid w_1, \dots, w_{t-1})$ (règle de la chaîne). On le forme

en minimisant l'entropie croisée moyenne ; la **perplexité** $PPL = \exp(-(1/T) \sum_t \log P(w_t \mid w_{<t}))$

mesure « le nombre moyen de choix » du modèle : plus elle est basse, mieux il prédit la suite.

3.2 Préparation des données séquentielles

Le corpus parallèle est tokenisé au niveau **mot** (minuscules), filtré par longueur et plafonné.

Deux

vocabulaires (source, cible) sont construits avec une fréquence minimale et les tokens spéciaux `<pad>`,

`<bos>`, `<eos>`, `<unk>`. Les mini-lots sont constitués avec **padding** et **masquage** (la perte ignore les positions `<pad>`).

3.3 Comparaison RNN / LSTM / GRU et gradient clipping

Cellule	Perplexité validation (à reporter)	Paramètres	Temps (s)
RNN	... (la plus élevée)	...	...
LSTM	...	...	...
GRU	... (bon compromis)	...	...

Tableau 6 — Comparaison des cellules récurrentes (à compléter depuis le notebook).

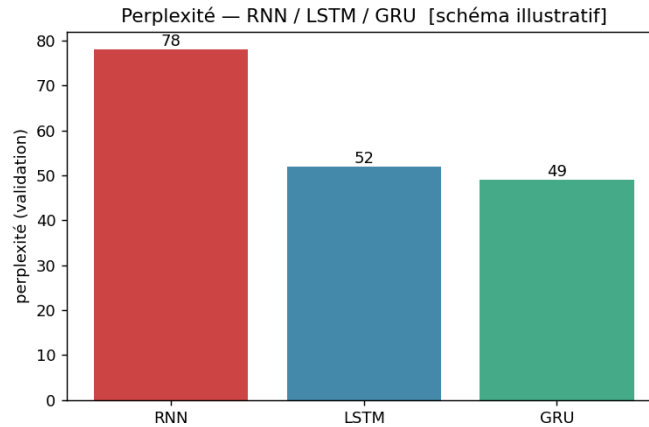


Figure 5 — Perplexité de validation des trois cellules (schéma illustratif). Le RNN simple présente typiquement la perplexité la plus élevée ; LSTM et GRU, dotés de portes, mémorisent mieux le contexte.

La **rétropropagation à travers le temps (BPTT)** peut faire exploser les gradients ; le ***gradient clipping*** borne leur norme. La Figure 6 illustre la stabilisation obtenue avec clipping.

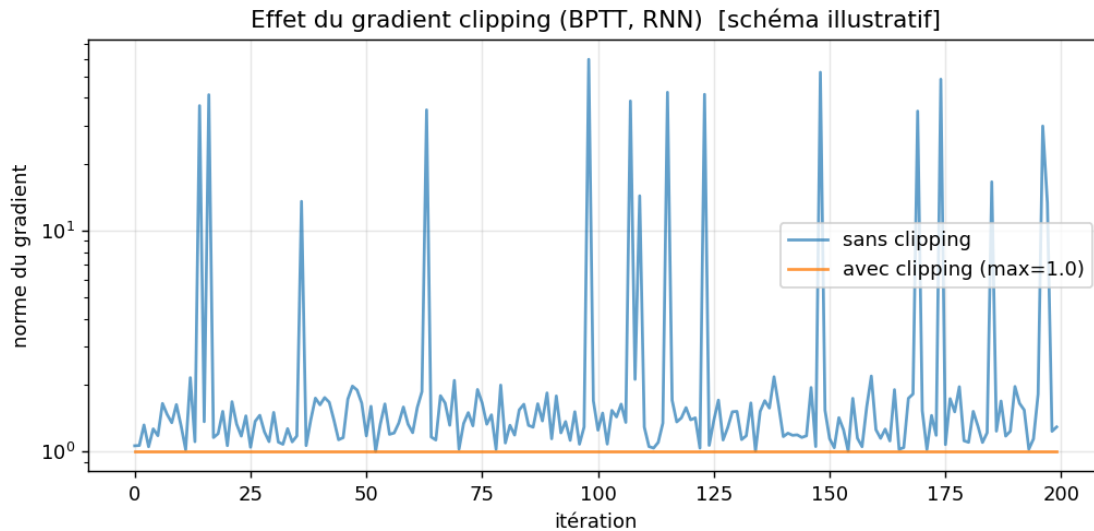


Figure 6 — Norme du gradient avec et sans clipping en BPTT (schéma illustratif). Sans clipping, des pics d'explosion apparaissent ; le clipping borne la norme et stabilise l'entraînement.

3.4 Système Seq2Seq et stratégies de décodage

Une architecture **encodeur–décodeur** à base de GRU, entraînée avec **teacher forcing**, réalise la traduction. Deux stratégies de décodage sont comparées : **glouton** et **beam search**.

Décodage	BLEU (à reporter)
Glouton	...
Beam search (largeur 3)	... (généralement $\geq$ glouton)

Tableau 7 — Score BLEU selon la stratégie de décodage.

3.5 Analyse critique et question de synthèse

La comparaison de perplexité confirme l'apport du *gating* (LSTM/GRU) pour mémoriser le contexte et

stabiliser le gradient. Avec un encodeur à état unique (sans attention) et un corpus réduit, la traduction

reste approximative : le goulot d'étranglement de l'état caché limite les phrases longues.

*Question de synthèse — Partie III. *Dans quelle mesure les architectures récurrentes modélisent-elles*

*efficacement une séquence, et comment justifier le passage RNN → LSTM/GRU → encodeur-décodeur ?**

Les RNN factorisent la probabilité d'une séquence token par token mais peinent sur les dépendances

longues ; les cellules à portes introduisent une mémoire régulée qui atténue ce problème. Pour la

transduction (traduction), le schéma encodeur-décodeur sépare la compréhension de la source de la

génération de la cible ; le décodage (glouton vs beam search) arbitre coût et qualité. Ces limites motivent

l'ajout de l'attention.

Synthèse — Partie III. Le biais inductif temporel des modèles récurrents, renforcé par le gating et

l'architecture encodeur-décodeur, permet de traiter des données séquentielles que ni le MLP ni le CNN ne

capturent.

Synthèse transversale

Les trois parties illustrent un même principe : ****l'architecture encode les a priori adaptés à la structure des**

données.**

Donnée	Structure exploitée	Architecture	Mécanisme clé
Tabulaire	aucune structure spatiale/temporelle	MLP	couches denses, interactions non linéaires
Image	dépendance locale + invariance par translation	CNN	localité, partage de poids, hiérarchie
Séquence	temporalité + dépendances à distance	RNN/LSTM/GRU/Seq2Seq	récurrence, mémoire à portes, encodeur–décodeur

Tableau 8 — Synthèse transversale des trois paradigmes.

Le paradigme reste l'apprentissage supervisé par descente de gradient, mais le **biais inductif** change : le

MLP ne suppose aucune structure, le CNN suppose la localité spatiale, les modèles récurrents supposent une

dépendance temporelle ordonnée. Choisir l'architecture, c'est **injecter la bonne hypothèse sur la donnée**,

ce qui réduit le nombre de paramètres nécessaires, accélère l'apprentissage et améliore la généralisation.

Lorsque ces hypothèses deviennent insuffisantes (dépendances très longues), on enrichit le modèle

(attention, Transformers), prolongeant la même logique d'adaptation.

Conclusion générale

Ce projet a relié théorie et pratique sur trois familles d'architectures de deep learning, à travers une

démarche commune : configuration centralisée, préparation rigoureuse des données, expérimentation

contrôlée et interprétation. Il met en évidence que la performance ne tient pas seulement à la profondeur du

réseau, mais à l'**adéquation entre l'architecture et la structure des données**. Le MLP traite des données

sans géométrie, le CNN exploite la localité spatiale, les modèles récurrents capturent la temporalité.

Limites. Pour rester exécutables sur Colab, les jeux de données sont sous-échantillonnés et les entraînements courts ; les performances absolues sont donc bornées. La Partie I n'inclut pas de validation

croisée, et le Seq2Seq de la Partie III est dépourvu de mécanisme d'attention.

Perspectives. Les évolutions naturelles incluent la validation croisée (Partie I), l'ajout de la *batch

normalization et de l'*augmentation de données (Partie II)*, et l'*intégration d'un mécanisme d'attention**

puis d'une architecture **Transformer** (Partie III) — généralisation directe de la logique d'adaptation de

l'architecture à la structure des données.

Annexe A — Configuration et reproductibilité

Chaque notebook débute par une cellule CONFIG (@dataclass) regroupant toute la configuration (source des données, découpages, graine aléatoire, *device*, hyperparamètres, chemins, plafonds de calcul). Une fonction `set_seed()` fixe l'aléa (`random`, `numpy`, `torch`) et `try_gpu()` sélectionne automatiquement le GPU. Modifier le comportement revient à modifier la seule cellule CONFIG.

Annexe B — Structure des livrables

```
deep learning/
|-- P1_MLP_HeartDisease.ipynb      # Partie I – MLP / tabulaire
|-- P2_CNN_EMNIST.ipynb           # Partie II – CNN / images
|-- P3_RNN_Seq2Seq_Tatoeba.ipynb  # Partie III – RNN / LSTM / GRU / Seq2Seq
|-- README.md                     # instructions d'exécution (Colab / local)
`-- RAPPORT.(md|pdf|docx)         # le présent rapport
```

Annexe C — Extraits de code significatifs

Sélection automatique du device et reproductibilité :

```
def set_seed(seed):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)

def try_gpu(i=0):
    if torch.cuda.is_available() and torch.cuda.device_count() >= i + 1:
        return torch.device(f"cuda:{i}")
    return torch.device("cpu")
```

Corrélation croisée 2D implémentée à la main (Partie II) :

```
def corr2d(X, K):
    h, w = K.shape
    Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i + h, j:j + w] * K).sum()
    return Y
```

Perplexité d'un modèle de langage (Partie III) :

```
@torch.no_grad()
def perplexity(model, loader, crit, device):
    model.eval(); tot, n = 0.0, 0
    for x, y in lm_batches(loader):
        x, y = x.to(device), y.to(device)
        loss = crit(model(x).reshape(-1, V), y.reshape(-1))
        ntok = (y != PAD).sum().item()
        tot += loss.item() * ntok; n += ntok
    return math.exp(tot / max(n, 1))
```

Annexe D — Guide d'exécution (Google Colab)

1. Ouvrir chaque notebook sur Google Colab.
2. Activer le GPU : *Exécution* → *Modifier* → *Paramètres du notebook* → *Accélérateur matériel* : *GPU*.
3. Exécuter toutes les cellules (*Exécution* → *Tout exécuter*).

Chaque notebook installe ses dépendances spécifiques via `%pip` (`ucimlrepo`, `sacrebleu`) ; `torch`, `torchvision`, `scikit-learn`, `pandas` et `matplotlib` sont déjà présents sur Colab. Les données sont

téléchargées automatiquement, avec des sources de secours en cascade pour la robustesse.